STAR Robot Control System

Comprehensive Technical Documentation

Complete Reference for Software Engineers

Version 1.0
December 2025

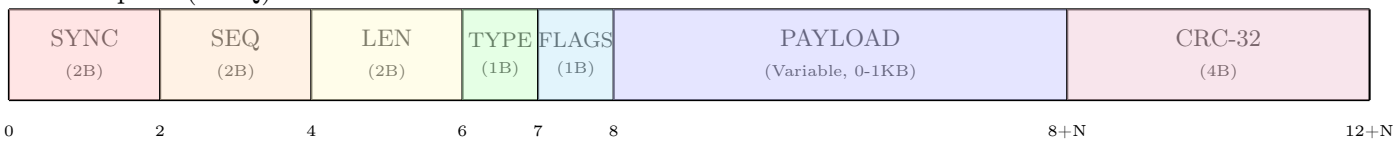
STAR Development Team
Locked Inc.

Contents

1 Nanopb (Protobuf) Protocol with ARQ and CRC for SPI Communication

1.1 Executive Summary

Overview: Protocol Buffers (nanopb) encapsulated in frames with CRC-32 and Automatic Repeat Request (ARQ) for reliable SPI communication between RPi5 and RX72N.



System Constraints:

- Renesas RX72N: 4MB flash, 1MB SRAM
- 100Hz SPI communication (10ms period)
- 250Hz motor control loop (4ms period)
- Static allocation only (no malloc)

1.1.1 Control Loop Frequency Rationale

Why different loop rates?

The nested loop architecture follows classic control systems design. The outer loop (SPI at 100Hz) handles command distribution and telemetry collection, while the inner loop (PID at 250Hz) responds quickly to motor dynamics. Running PID at 2.5 times the communication rate ensures smooth control between commands without wasting CPU on excessive computation.

Why 250Hz for motor control?

This is a Nyquist sampling consideration. With DC gearmotors at 210 RPM (3.5 Hz mechanical speed) and a first-order time constant $\tau = 75$ ms (cutoff frequency $f_c = 1/(2\pi\tau) \approx 2.1$ Hz), the 250Hz control rate provides approximately 60 times oversampling of the mechanical dynamics - well above the 2 times Nyquist minimum.

Running at 250Hz balances:

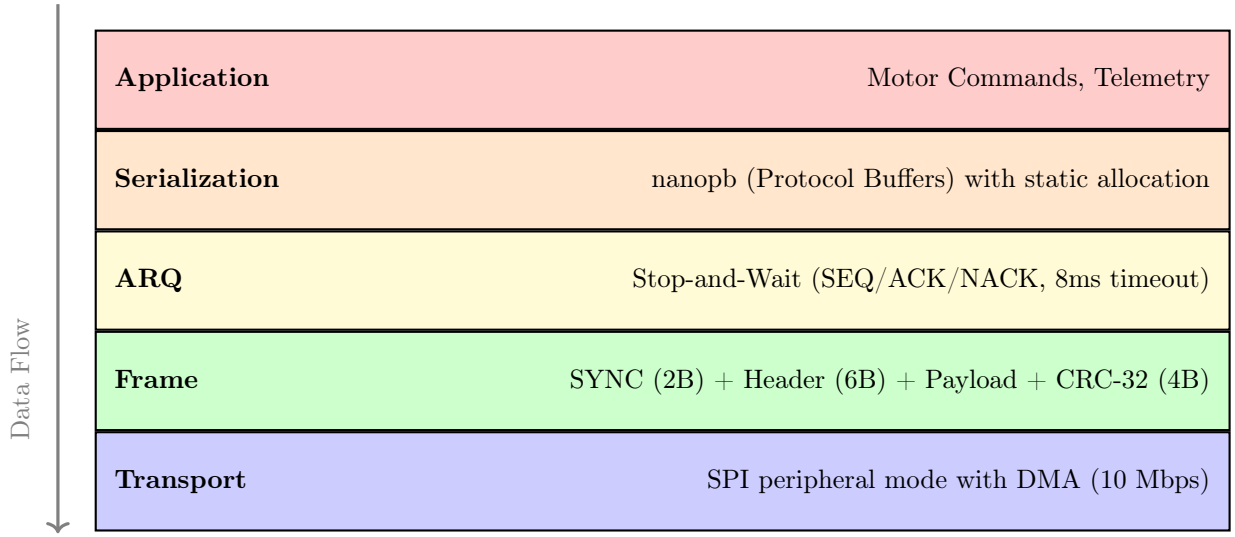
- Adequate sampling of motor dynamics (60 times oversampling)
- Efficient CPU utilization (4 motors $\times$ PID + encoder reads)
- Low latency between velocity commands and motor response
- Power efficiency (fewer ADC conversions, GPIO reads, and computations)

For context, control loop frequencies should match system dynamics:

- DC gearmotors: 50-500 Hz typical
- High-speed servos: 1-10 kHz
- Motor current control: 10-50 kHz
- Temperature control: 0.1-10 Hz

1.2 Key Design Decisions

1.2.1 Protocol Stack



1.2.2 Byte Order and Standards Compliance

The protocol follows established standards for byte ordering to ensure cross-platform compatibility:

Table 1: Byte Order Standards by Protocol Field

Field	Byte Order	Standard	Rationale
SYNC	Big-endian	RFC 1700	Network byte order for headers
SEQ	Big-endian	RFC 1700	Network byte order for headers
LEN	Big-endian	RFC 1700	Network byte order for headers
TYPE	N/A (1 byte)	–	Single byte, no endianness
FLAGS	N/A (1 byte)	–	Single byte, no endianness
CRC-32	Little-endian	IEEE 802.3	Standard CRC-32 polynomial

SYNC Magic Word (0x55AA): The frame synchronization marker uses the distinctive bit pattern 0x55AA (binary: 0101010101010101 in big-endian wire format). This alternating pattern is easily recognizable and unlikely to appear in random data, making it ideal for frame alignment. The value 0x55AA is transmitted as bytes `[0x55, 0xAA]` on the wire.

RFC 1700 (Network Byte Order): All multi-byte header fields (SYNC, SEQ, LEN) use big-endian format following RFC 1700 "Assigned Numbers" standard. This ensures compatibility when communicating between different architectures (ARM RPi5 and Renesas RX72N, both little-endian internally).

IEEE 802.3 (CRC-32): The frame checksum uses IEEE 802.3 polynomial (0x04C11DB7) in little-endian format, compatible with standard Ethernet CRC libraries. This allows zero-copy CRC validation using hardware accelerators where available.

Key Insight: Mixed byte orders are intentional — headers follow network standards for cross-platform compatibility, while CRC follows IEEE 802.3 for hardware acceleration and library compatibility.

1.2.3 Memory Budget

Table 2: SRAM Usage (512 KB total)

Component	Size (KB)	Usage (%)
DMA double-buffer	8.0	1.56
RX/TX buffers	2.0	0.39
Protobuf structs	1.5	0.29
ARQ state	4.0	0.78
Total	**15.5**	**3.03**

Table 3: Flash Usage (16 MB total)

Component	Size (KB)	Usage (%)
Generated protobuf	80	0.49
Nanopb runtime	12	0.07
Frame + ARQ layers	8	0.05
Total	**100**	**0.61**

Memory Budget Calculations:

SRAM (512 KB total):

- **DMA double-buffer:** 2×4092 bytes $= 8184$ bytes $= 8.0$ KB
  Calculation: Two ping-pong DMA buffers at `STAR_SPI_DMA_MAX_SIZE` (4092 bytes each)
  Usage: $\frac{8.0}{512} \times 100 = 1.56\%$
- **RX/TX buffers:** $1024 + 1024 = 2048$ bytes $= 2.0$ KB
  Calculation: 1 KB RX buffer + 1 KB TX buffer for staging protobuf messages
  Usage: $\frac{2.0}{512} \times 100 = 0.39\%$
- **Protobuf structs:** 1536 bytes $= 1.5$ KB
  Calculation: Largest message structures (SetVelocityResponse $\approx$ 600 bytes, TelemetryData $\approx$ 400 bytes)
  Usage: $\frac{1.5}{512} \times 100 = 0.29\%$
- **ARQ state:** $4 + 4 + 8 + 4 + 4092 = 4112$ bytes ≈ 4.0 KB
  Calculation: Sequence number (4B) + retry counter (4B) + timestamp (8B) + state (4B) + retry buffer (4092B)
  Usage: $\frac{4.0}{512} \times 100 = 0.78\%$
- **Total SRAM:** $8.0 + 2.0 + 1.5 + 4.0 = 15.5$ KB
  Total usage: $\frac{15.5}{512} \times 100 = 3.03\%$

Flash (16 MB = 16384 KB total):

- **Generated protobuf:** 80 KB
  Calculation: 6 proto files (common, motor_control, telemetry, battery_management, configuration, firmware_update) $\times$ 13 KB avg compiled size
  Usage: $\frac{80}{16384} \times 100 = 0.49\%$
- **Nanopb runtime:** 12 KB
  Calculation: Compiled size of pb_encode.c, pb_decode.c, pb_common.c
  Usage: $\frac{12}{16384} \times 100 = 0.07\%$
- **Frame + ARQ layers:** 8 KB
  Calculation: Framing logic + CRC-32 + ARQ state machine + retransmission logic
  Usage: $\frac{8}{16384} \times 100 = 0.05\%$
- **Total Flash:** $80 + 12 + 8 = 100$ KB
  Total usage: $\frac{100}{16384} \times 100 = 0.61\%$

1.3 Implementation Requirements

1.3.1 Protocol Stack Architecture

The protocol is implemented as a layered stack on both the RPi5 (SPI controller) and RX72N (SPI peripheral). Each layer has specific responsibilities:

Layer	Responsibility	Implementation
5. Application	Motor commands, telemetry, control logic	Different per platform
4. Protobuf	Message encode/decode (nanopb)	Both sides
3. ARQ	Sequence numbering, ACK/NACK, retry logic	Both sides (symmetric)
2. Framing	Header/footer, CRC-32 verification	Both sides (identical)
1. SPI+DMA	Physical transport with double-buffering	Both sides (different roles)

Table 4: Protocol stack layers and implementation requirements

Key Insight: Layers 2-4 are nearly identical on both platforms. Only the application layer (Layer 5) and SPI role (Layer 1: controller vs peripheral) differ.

1.3.2 RX72N (SPI Peripheral) Implementation

Layer 1: SPI Peripheral with DMA

- Configure RSPI in peripheral mode
- Set up DMA with double-buffering: 2×4092 bytes
- Register DMA interrupt callbacks for buffer swapping
- Wait for chip select from RPi5, respond to clock

Layer 2: Frame Layer (Custom Code)

RX Path:

- Parse frame header (magic bytes, length, sequence number)
- Verify CRC-32 over header + payload
- If CRC fails → discard packet, send NACK
- If CRC valid → extract payload, pass to ARQ layer

TX Path:

- Build frame: `[Header | Payload | CRC32 | Footer]`
- Calculate CRC-32 over header + payload
- Queue complete frame to DMA buffer for transmission

Layer 3: ARQ Layer (Custom Code)

RX Path:

- Check sequence number against `last_acked_rx`
- If duplicate (`seq` $\leq$ `last_acked_rx`) $\rightarrow$ send ACK, discard payload
- If expected (`seq` = `last_acked_rx` + 1) $\rightarrow$ accept, send ACK
- If out-of-order $\rightarrow$ send NACK, request retransmission

TX Path:

- Assign `tx_sequence_number` to outgoing frame
- Store frame in retry buffer
- Start timeout timer (e.g., 10ms)
- On timeout $\rightarrow$ retransmit (up to 3 attempts)
- On ACK received $\rightarrow$ clear retry buffer, increment `tx_seq`

Layer 4: Protobuf (nanopb library)

- Decode incoming bytes $\rightarrow$ `SetVelocityRequest`, `EmergencyStopRequest`
- Encode outgoing structs $\rightarrow$ `SetVelocityResponse`, `TelemetryData`
- Uses `star.v1` package following Boston Dynamics style guide
- Static allocation (no malloc) with `.options` file constraints

Layer 5: Application

- Handle motor velocity commands
- Read encoder data via multiplexer
- Execute PID control loop (250 Hz per motor)
- Send telemetry responses (encoder feedback, current, temperature)

1.3.3 RPi5 (SPI Controller) Implementation

Layer 1: SPI Controller with DMA

- Use Linux `/dev/spidev` interface
- Configure SPI3: 10 Mbps, mode 0, 8-bit words
- Use `ioctl(SPI_IOC_MESSAGE)` for DMA transfers
- Implement userspace double-buffering for continuous operation

Layer 2: Frame Layer *(Custom Code - SAME AS RX72N)*

RX Path: Parse frame header, verify CRC-32, extract payload or discard on CRC failure

TX Path: Build frame `[Header | Payload | CRC32 | Footer]`, calculate CRC-32, send via SPI

Layer 3: ARQ Layer *(Custom Code - SAME AS RX72N)*

RX Path: Check sequence number, detect duplicates $\rightarrow$ ACK and discard, detect out-of-order $\rightarrow$ NACK

TX Path: Assign sequence number, store in retry buffer, timeout and retry logic, handle ACKs

Layer 4: Protobuf (Go with protoc-gen-go)

- Encode: `SetVelocityRequest`, `EmergencyStopRequest`
- Decode: `SetVelocityResponse`, `TelemetryData`, `EncoderData`
- Uses `star.v1` package following Boston Dynamics style guide

Layer 5: Application (star-gateway)

- Send motor velocity commands from Nav2 (autonomous) or UI (manual)
- All control flows through Nav2 stack for consistent behavior
- Receive encoder feedback for odometry calculations
- Bridge ROS2 topics to/from RX72N via SPI

1.3.4 ARQ Symmetry and State Management

The ARQ layer uses **Stop-and-Wait** protocol (also called alternating bit protocol). This means each side sends one frame at a time, waits for acknowledgment (ACK) with a timeout, and retries up to 3 times before declaring failure. This approach provides:

- Simple implementation suitable for embedded systems
- Predictable worst-case latency for real-time control
- Low memory overhead (single retry buffer, not a window of frames)
- Sufficient throughput for 10 Mbps SPI with <1ms frame time

The ARQ protocol is **symmetric** — both sides implement identical Stop-and-Wait logic but maintain separate state for their own transmissions and receptions.

Each Side Maintains:

- **TX sequence number** (`tx_seq`): For packets it sends
- **RX last acknowledged** (`rx_last_ack`): For packets it receives
- **Retry buffer**: Stores unacknowledged outgoing packets
- **Timeout timer**: Triggers retransmission if no ACK received

Responsibility	RX72N (Peripheral)	RPi5 (Controller)
Sends	Telemetry, responses	Commands, requests
TX sequence numbering	Own `tx_seq`	Own `tx_seq`
ACK/NACK generation	Sends ACK for RPi5	Sends ACK for RX72N
Retry logic	Retries if no ACK	Retries if no ACK
Duplicate detection	Checks RPi5's `seq`	Checks RX72N's `seq`

Table 5: ARQ role distribution between controller and peripheral

Important: The SPI controller/peripheral roles (Layer 1) are **hardware-level only**. The ARQ layer (Layer 3) treats both sides as equals — each can send, ACK, and retry independently.

1.3.5 Example Transaction: RPi5 Sends Velocity Command

1. **RPi5 Application:** Create `SetVelocityRequest` (left=1.0 m/s, right=1.0 m/s)
2. **RPi5 Protobuf:** Encode to binary (nanopb)
3. **RPi5 ARQ:** Assign `seq=42`, save to retry buffer, start 10ms timeout
4. **RPi5 Frame:** Build packet with header, CRC-32, footer
5. **RPi5 SPI:** DMA transfer to RX72N (controller initiates)
6. **RX72N SPI:** DMA receives packet in double-buffer
7. **RX72N Frame:** Verify CRC-32 → valid
8. **RX72N ARQ:** Check `seq=42` = `rx_last_ack+1` → accept
9. **RX72N Protobuf:** Decode to `SetVelocityRequest` struct
10. **RX72N Application:** Update motor setpoints, execute PID control
11. **RX72N ARQ:** Build ACK packet with `ack_seq=42`
12. **RX72N Frame:** Wrap ACK in frame with CRC-32
13. **RX72N SPI:** DMA transmits ACK (peripheral responds)
14. **RPi5 SPI:** DMA receives ACK packet
15. **RPi5 Frame:** Verify CRC-32 → valid
16. **RPi5 ARQ:** Got `ACK(42)` → clear retry buffer, cancel timeout

Outcome: Command successfully delivered with guaranteed delivery via ARQ. If ACK was lost or CRC failed, RPi5 would retransmit after 10ms timeout.

1.4 ARQ Communication Flow

1.4.1 Normal Operation (No Retransmission)

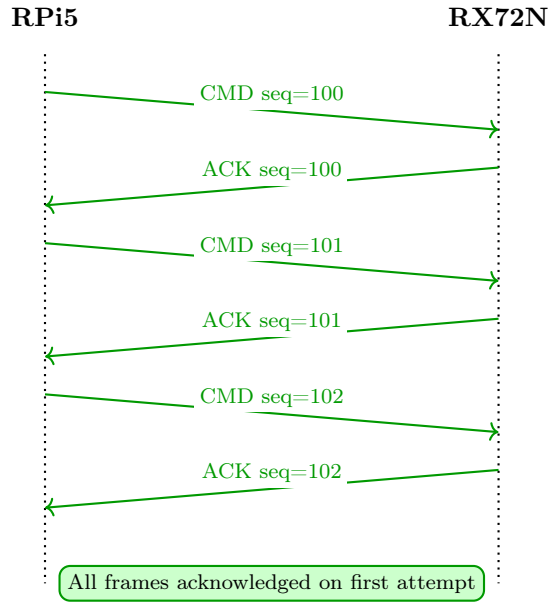


Figure 1: Normal ARQ Flow - Three Successful Transactions

1.4.2 Error Recovery Scenarios

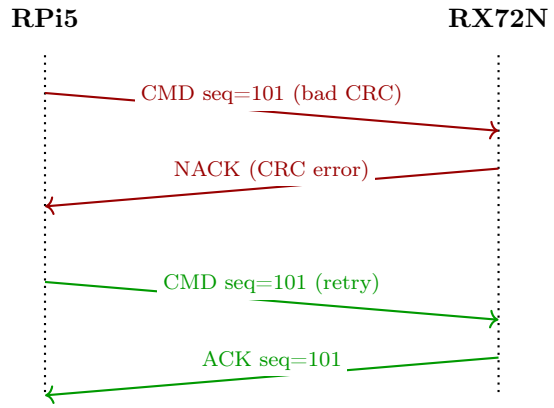


Figure 2: CRC Error - Retry with Same Sequence Number

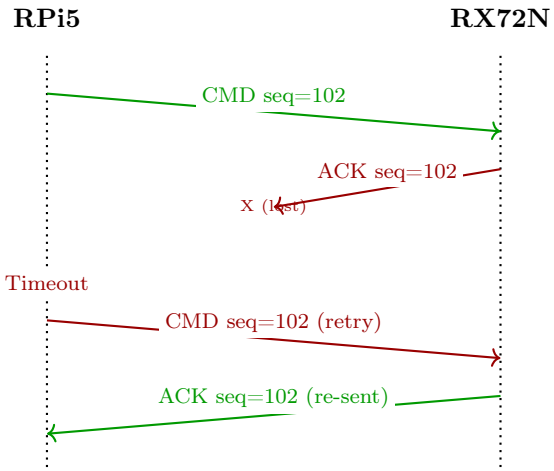


Figure 3: Duplicate Packet Scenario (Lost ACK)

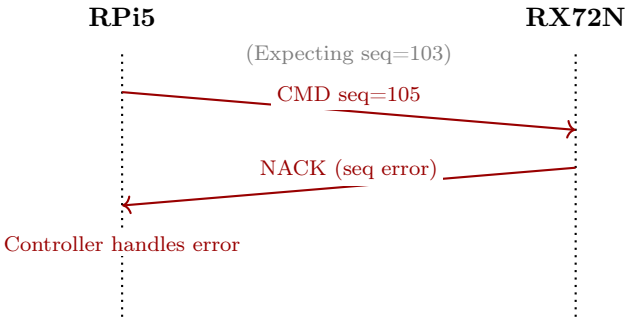


Figure 4: Out-of-Order Packet Scenario

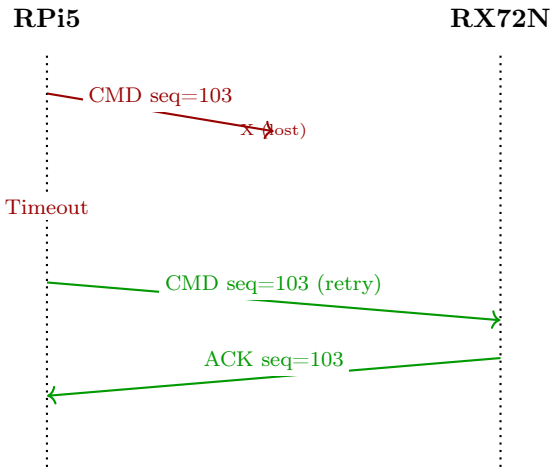


Figure 5: Lost Packet Scenario (Timeout)

1.5 Error Handling Strategies

CRC Mismatch
Action: NACK with CRC error reason
Recovery: Controller retry
Logging: Increment crc_errors

Sequence Error
Duplicate: Re-send ACK
Out-of-order: NACK + reject
Recovery: Controller retry

Timeout
RX timeout: Continue next cycle
500ms watchdog: Emergency stop
Retry fail: Log error

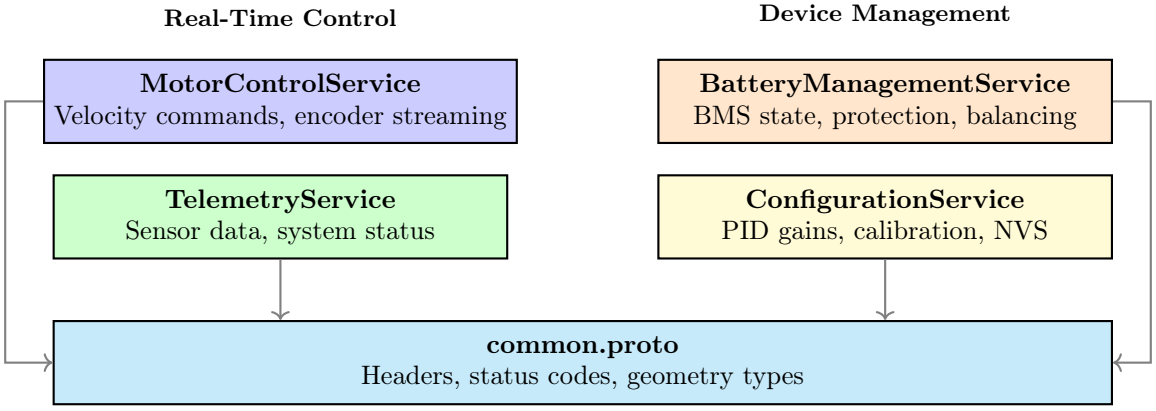
Invalid Protobuf
Action: NACK protocol error
Validate: IDs, ranges, offsets
Logging: Increment proto_errors

2 Protocol Buffer Schema Architecture

This section documents the Protocol Buffer schema architecture for RPi5↔RX72N communication. All schemas follow the **Boston Dynamics Protobuf Style Guide** for consistency, maintainability, and industry best practices.

2.1 Architecture Overview

Overview: Five gRPC services define the complete API surface for robot control, organized by functional domain. Services communicate via the nanopb-serialized frames described in Section 1.



Key Design Principles

- **Versioned Package:** All services use `star.v1` namespace for API evolution
- **Request/Response Headers:** Every RPC includes tracing, timestamps, and status
- **Streaming Support:** Continuous data uses server streaming (up to 100Hz)
- **Static Allocation:** All messages sized for nanopb constraints (no dynamic memory)

Note: Naming conventions, unit suffixes, and coding standards are consolidated in Section 4 (Style Guide and Conventions).

2.2 Service Specifications

2.2.1 MotorControlService

Purpose: Low-latency differential drive control with real-time encoder feedback.

Target Latency: <10ms round-trip for velocity commands.

Table 6: MotorControlService RPCs

RPC	Type	Description
SetVelocity	Unary	Set left/right wheel velocities (m/s)
EmergencyStop	Unary	Immediate stop, priority over all commands
SetMotorPower	Unary	Direct PWM control (bypass PID, testing only)
StreamEncoders	Server stream	Encoder ticks and velocity at 1-100Hz
ControlStream	Bidirectional	Velocity in, encoder feedback out

Key Messages:

- `VelocityCommand` – Left/right velocity (m/s), sequence number, timestamp
- `EncoderData` – Motor ID, tick count (341.2 PPR), velocity, timestamp
- `MotorStatus` – Duty cycle, velocity, temperature, current, fault flags
- `PidConfig` – Kp, Ki, Kd gains with output saturation limits

2.2.2 TelemetryService

Purpose: Real-time sensor data aggregation and system health monitoring.

Table 7: TelemetryService RPCs

RPC	Type	Description
GetTelemetry	Unary	Snapshot of all sensor data
StreamTelemetry	Server stream	Continuous sensor updates at 1-100Hz
GetSystemStatus	Unary	Firmware version, uptime, connection states

Key Messages:

- `TelemetryData` – IMU, GPS, battery %, WiFi signal, CPU usage, temperature
- `ImuData` – Pitch/roll/yaw (rad), acceleration (m/s^2), angular velocity (rad/s)
- `GpsData` – Lat/lon (degrees), altitude (m), accuracy, satellite count, fix type
- `SystemStatus` – Connection states, operating mode, free heap, uptime

Enumerations:

- `RobotMode` – IDLE, MANUAL, AUTONOMOUS, MAPPING, EMERGENCY_STOP
- `ConnectionStatus` – CONNECTED, DISCONNECTED, CONNECTING, ERROR
- `GpsFix` – NONE, 2D, 3D, DGPS, RTK_FLOAT, RTK_FIXED

2.2.3 BatteryManagementService

Purpose: BQ7850 BMS monitoring for lithium-ion battery pack safety and management.
Hardware: TI BQ7850 16-cell BMS with I2C interface.

Table 8: BatteryManagementService RPCs

RPC	Type	Description
GetBatteryState	Unary	Complete battery snapshot (cells, temps, SOC)
StreamBatteryState	Server stream	Continuous battery monitoring
Get/SetProtectionThresholds	Unary	OV/UV/OC/OT protection limits
Enable/DisableCellBalancing	Unary	Per-cell passive balancing control
ControlFets	Unary	Charge/discharge FET enable/disable
GetDeviceInfo	Unary	Manufacturer, serial, chemistry, versions

Key Messages:

- `BatteryState` – Composite of all battery data
- `CellData` – Individual cell voltages (mV), pack voltage, valid cell count
- `BatteryTemperatureData` – Thermistor readings (0.1°C), average
- `StateOfChargeData` – Relative/absolute SOC (%), capacity (mAh), cycle count
- `ProtectionThresholds` – OV/UV voltage limits, OC current limits, OT temperatures

2.2.4 ConfigurationService

Purpose: Runtime parameter management with NVS persistence across power cycles.

Table 9: ConfigurationService RPCs

RPC	Type	Description
GetConfiguration	Unary	Read current system configuration
SetConfiguration	Unary	Apply configuration (optional NVS persist)
ResetToDefaults	Unary	Restore factory defaults
ValidateConfiguration	Unary	Validate without applying
SaveConfiguration	Unary	Persist in-memory config to NVS
Get/SetMotorPidConfig	Unary	Per-motor PID tuning

Key Messages:

- `SystemConfiguration` – Complete config with version and CRC32
- `MotorPidConfig` – Per-motor PID gains (4 motors supported)
- `CurrentSensorCalibration` – Offset and scale for current sensing
- `SafetyThresholds` – Overcurrent, thermal shutdown, cell imbalance limits
- `TimingConfig` – Motor period, telemetry period, communication timeout

2.3 common.proto – Shared Types

All services import `common.proto` for standardized request/response handling.

Table 10: Standard Headers

Message	Fields
RequestHeader	request_id (UUID), client_timestamp, client_version, timeout
ResponseHeader	request_id (echo), server_timestamp, status, error_message, latency_us

Status Codes: UNKNOWN, OK, INVALID_REQUEST, INTERNAL_ERROR, NOT_FOUND, TIMEOUT, INVALID_STATE, ESTOP_ACTIVE

Geometric Types:

- `Vector3` – Generic 3D vector (x, y, z)
- `Quaternion` – Orientation (w, x, y, z)
- `SE2Pose` – 2D pose (x_m, y_m, angle_rad)
- `SE2Velocity` – 2D velocity (vx_mps, vy_mps, omega_rad_per_s)

2.4 Code Generation and Integration

Table 11: Code Generation Targets

Target	Generator	Output	Usage
Go	protoc-gen-go + grpc	`gen/go/`	star-gateway (RPi5)
nanopb	nanopb_generator	`gen/nanopb/`	RX72N firmware

nanopb Constraints: RX72N requires static allocation. Field sizes are configured in `.options` files:

- String fields: `max_size:64` (request_id), `max_size:128` (error_message)
- Repeated fields: `max_count:16` (cell voltages), `max_count:4` (motors)
- Bytes fields: `max_size:1024` (firmware chunks)

Build Integration:

- **Linting:** `buf lint` enforces style guide compliance
- **Breaking Changes:** `buf breaking` detects API incompatibilities
- **CI Pipeline:** Automatic generation and testing on PR

3 Hardware Pin Connections

3.1 RX72N Microcontroller Pinout (R5F572NNHGFP#30)

Color Legend:

Function Category	Color
PWM Motor Control (GPTW)	Blue
Encoder Inputs (MTU)	Green
SPI Communication	Orange
SMBUS (BMS)	Violet
I2C (Charger)	Teal
ADC Current Sensing	Red
Debug/JTAG/Boot	Yellow
Power/Ground	Gray
Status LEDs	Magenta
Temperature/nFAULT	Lime
PMOD JA (SPI/Display)	Olive
PMOD JB (I2C Sensors)	Dark Brown
PMOD JC (UART/Wireless)	Bright Cyan
Motor Driver SPI (Config)	Purple
Unused/Available	White

Pin	Pin Name	Connected To	Note/Comment
1	AVCC1	Bypass caps	Power supply (analog)
2	EMLE	Emulator Configuration jumper	See Table 16
3	AVSS1	Ground	Power supply (analog)
4	PJ3	Unused/Available	Not connected (has SCI/SPI/SSI/Ethernet if needed)
5	VCL	220nF cap to ground	Voltage regulator
6	VBATT	Boards 3v3	Battery backup
7	MD/FINED	E1E2-Emulator-Interface, DIP switch	See Table 17. Default: OFF,OFF
8	XCIN	10k resistor to ground	RTC crystal input (not used)
9	XCOUT	Floating	RTC crystal output (not used)
10	RES#	E1E2-Emulator-Interface, Reset button	MCU reset
11	P37/XTAL	24MHz crystal (to pin 13)	System clock crystal
12	VSS	Ground	Power supply
13	P36/EXTAL	24MHz crystal (to pin 11)	System clock crystal
14	VCC	Boards 3v3	Power supply
15	P35/NMI/UPSEL	UPSEL configuration jumper	Can be used for NMI (not used). See Table 18
16	P34/TST#	E1E2-Emulator-Interface	JTAG (TRST#)
17	P33	RES#, CY7C65213A-28PVXI	USB to UART IC for flashing (resets system before firmware flash)

Continued on next page

Pin	Pin Name	Connected To	Note/Comment
18	P32	Motor Driver SPI nCS2	RSPIC chip select for Motor 1 (DRV8243S)
19	P31/TMS	E1E2-Emulator-Interface	JTAG (TMS)
20	P30/TDI	E1E2-Emulator-Interface	JTAG (TDI)
21	P27/TCK	E1E2-Emulator-Interface	JTAG (TCK)
22	P26/TDO	E1E2-Emulator-Interface	JTAG (TDO)
23	P25/MTCLKB	Encoder 3 Phase A	MTU4 encoder (quadrature counting)
24	P24/MTCLKA	Encoder 0 Phase A	MTU1 encoder (quadrature counting)
25	P23	Motor Driver SPI nCS1	RSPIC chip select for Motor 0 (DRV8243S)
26	P22/MTCLKC	Encoder 1 Phase B	MTU2 encoder (quadrature counting)
27	P21/SCL1	Charger I2C Clock	RIIC1 for battery charger
28	P20/SDA1	Charger I2C Data	RIIC1 for battery charger
29	P17	Motor Driver SPI nCS0	RSPIC chip select for Motor 0 (DRV8243S)
30	P16/USB0_VBUS	USB0 VBUS	Optional USB boot
31	P15/MTCLKB	Encoder 0 Phase B	MTU1 encoder (quadrature counting)
32	P14/MTCLKA	Encoder 1 Phase A	MTU2 encoder (quadrature counting)
33	P13/SMBD0	BMS SMBUS Data	RIIC0 for BMS (FM+ capable)
34	P12/SMBC0	BMS SMBUS Clock	RIIC0 for BMS (FM+ capable)
35	VCC_USB	Boards 3v3	USB power supply
36	USB0_DM	USB0 DN (27R resistor)	USB data -
37	USB0_DP	USB0 DP (27R resistor)	USB data +
38	VSS_USB	Ground	USB ground
39	P55	PMOD JB GPIO1	See Table 14
40	P54	PMOD JB GPIO2	See Table 14
41	P53	PMOD JA SCK	SPI Clock. See Table 13
42	P52	PMOD JA CS0	SPI Chip Select 0. See Table 13
43	P51	PMOD JA CIPO	SPI Controller In. See Table 13
44	P50	PMOD JA COPI	SPI Controller Out. See Table 13
45	PC7/UB	E1E2-Emulator-Interface, DIP switch	Operation mode configuration. See Table 17. Also GTIOC3A alternate
46	PC6	PMOD JB GPIO0	See Table 14
47	PC5	PMOD JB INT2	I2C Interrupt 2. See Table 14

Continued on next page

Pin	Pin Name	Connected To	Note/Comment
48	PC4	PMOD JB INT1	I2C Interrupt 1. See Table 14
49	PC3	PMOD JB SDA	I2C Data. See Table 14
50	PC2	PMOD JB SCL	I2C Clock. See Table 14
51	PC1/MTIOC3A	Encoder 2 Phase A	MTU3 encoder (quadrature counting)
52	PC0/MTIOC3C	Encoder 2 Phase B	MTU3 encoder (quadrature counting)
53	PB7/TXD9	Debug SCI USB-C RX	UART crossed. See Table 20
54	PB6/RXD9	Debug SCI USB-C TX	UART crossed. See Table 20
55	PB5	PMOD JA CS1	SPI Chip Select 1. See Table 13
56	PB4	PMOD JA DC	Data/Command (displays). See Table 13
57	PB3	PMOD JA RST	Reset signal. See Table 13
58	PB2	PMOD JA GPIO	Extra GPIO. See Table 13
59	PB1	PMOD JC GPIO2	See Table 15
60	VCC	Boards 3v3	Power supply
61	PB0	PMOD JC GPIO3	See Table 15
62	VSS	Ground	Power supply
63	PA7/MISOA-B	RPi5 RSPIA CIPO	RSPIA-B Controller In Peripheral Out
64	PA6/MOSIA-B	RPi5 RSPIA COPI	RSPIA-B Controller Out Peripheral In
65	PA5/RSPCKA-B	RPi5 RSPIA SCLK	RSPIA-B clock
66	PA4/SSLA0-B	RPi5 RSPIA CS	RSPIA-B chip select
67	PA3/MTCLKD	Encoder 3 Phase B	MTU4 encoder (quadrature counting)
68	PA2	PMOD JB GPIO3	See Table 14
69	PA1	Unused/Available	Not connected (has SCK5, GTIOC2A, IRQ11 if needed)
70	PA0	Motor Driver SPI nCS3	RSPIC chip select for Motor 3 (DRV8243S)
71	PE7/GTIOC3A	Motor 3 PH (Phase)	GPTW PWM channel 3A (32-bit)
72	PE6/GTIOC3B	Motor 3 EN (Enable)	GPTW PWM channel 3B (32-bit)
73	PE5/GTIOC0A	Motor 0 PH (Phase)	GPTW PWM channel 0A (32-bit)
74	PE4/GTIOC1A	Motor 1 PH (Phase)	GPTW PWM channel 1A (32-bit)
75	PE3/GTIOC2A	Motor 2 PH (Phase)	GPTW PWM channel 2A (32-bit)
76	PE2/GTIOC0B	Motor 0 EN (Enable)	GPTW PWM channel 0B (32-bit)

Continued on next page

Pin	Pin Name	Connected To	Note/Comment
77	PE1/GTIOC1B	Motor 1 EN (Enable)	GPTW PWM channel 1B (32-bit)
78	PE0/GTIOC2B	Motor 2 EN (Enable)	GPTW PWM channel 2B (32-bit)
79	PD7	PMOD JC UART_TX	UART Transmit. See Table 15
80	PD6	PMOD JC UART_RX	UART Receive. See Table 15
81	PD5	PMOD JC RTS	Request to Send. See Table 15
82	PD4	PMOD JC CTS	Clear to Send. See Table 15
83	PD3	Motor Driver SPI SCLK	RSPIC clock (RSPCKC-A) to all 4 DRV8243S drivers
84	PD2	Motor Driver SPI CIPO	RSPIC controller in (MISOC-A) from DRV8243S
85	PD1	Motor Driver SPI COPI	RSPIC controller out (MOSIC-A) to DRV8243S
86	PD0	Status LED	User status indicator (active high, can also use IRQ0 or ADC AN108)
87	P47/IRQ15-DS	Motor 3 nFAULT	DRV8243S fault signal (active low, interrupt-capable)
88	P46/IRQ14-DS	Motor 2 nFAULT	DRV8243S fault signal (active low, interrupt-capable)
89	P45/IRQ13-DS	Motor 1 nFAULT	DRV8243S fault signal (active low, interrupt-capable)
90	P44/IRQ12-DS	Motor 0 nFAULT	DRV8243S fault signal (active low, interrupt-capable)
91	P43/AN003	Motor 3 Current Sense	ADC Unit 0, 12-bit, 2.0A max (IPROPI from DRV8243S)
92	P42/AN002	Motor 2 Current Sense	ADC Unit 0, 12-bit, 2.0A max (IPROPI from DRV8243S)
93	P41/AN001	Motor 1 Current Sense	ADC Unit 0, 12-bit, 2.0A max (IPROPI from DRV8243S)
94	VREFL0	Ground	ADC reference low
95	P40/AN000	Motor 0 Current Sense	ADC Unit 0, 12-bit, 2.0A max (IPROPI from DRV8243S)
96	VREFH0	3.3V	ADC reference high
97	AVCC0	Boards 3v3	ADC power supply
98	P07	Status LED	Status indicator
99	AVSS0	Ground	ADC ground
100	P05	Error LED	Error indicator

3.2 Motor Control Architecture

3.2.1 GPTW vs MTU Selection for PWM Generation

The RX72N provides two timer peripherals suitable for motor control PWM generation: the General PWM Timer (GPTW) and the Multi-Function Timer Unit (MTU3a). After detailed analysis,

GPTW was selected for the following technical reasons:

Resolution and Performance:

- **GPTW:** 32-bit timer resolution enabling precise duty cycle control
- **MTU3a:** 16-bit timer resolution (65,536 maximum count)
- At 20 kHz PWM frequency with 120 MHz peripheral clock:
  - GPTW provides 6,000 discrete duty cycle steps (120 MHz / 20 kHz)
  - MTU limited to maximum 3,276 steps before counter overflow

Channel Architecture:

- **GPTW:** 4 independent channels, each with 2 complementary outputs (GTIOC0A/B through GTIOC3A/B)
- **Perfect match** for 4 motors in Phase/Enable (PH/EN) control mode
- Each motor driver (DRV8243S) requires exactly 2 PWM signals

Peripheral Separation:

- Using GPTW for PWM and MTU for encoders provides clean functional separation
- MTU channels (MTU1/2/3/4) dedicated to encoder quadrature counting
- No resource conflicts between PWM generation and encoder reading

Pin Flexibility:

- GPTW signals available on multiple pin locations via MPC (Multi-Function Pin Controller)
- Example: GTIOC0A available on pins 25, 65, 73, 83
- Enabled conflict-free allocation by selecting alternate pin groups

3.2.2 Pin Allocation Strategy

Design Philosophy:

1. **Peripheral Grouping:** Place related functions on contiguous port pins
   - GPTW PWM: PE0-PE7 (8 consecutive pins, pins 71-78)
   - SPI to RPi5: PA4-PA7 (4 adjacent pins for optimal PCB routing)
   - MTU Encoders: P14-P15, P22-P27 (P2X port group)
   - I2C/SMBUS: P12-P13 (RIIC0), P20-P21 (RIIC1)
2. **Conflict Resolution via Alternate Functions:**
   - Initial pin selection caused conflicts between GPTW, I2C, and SPI
   - Table 1.10 analysis revealed all peripherals have 2-4 alternate pin locations
   - Consolidated all GPTW signals to Port E (PE0-PE7) for continuous block
   - Freed PA0-PA2 for future expansion (3 additional GPIO pins)

- Freed critical pins for I2C (P20-P21) and encoders (P22-P27)

3. **PCB Routing Optimization:**
   - Contiguous pin blocks minimize trace crossings
   - All 8 motor PWM signals on consecutive pins (PE0-PE7, pins 71-78)
   - SPI signals on PA4-PA7 enable compact layout
   - Simplified PCB layout with Port E dedicated to motor control
4. **Future Expandability:**
   - 35 GPIO pins available after critical allocations
   - PA0-PA2 freed by continuous GPTW block design
   - Unused peripherals documented: Ethernet, QSPI, SDHI, additional SCI/RSPI channels
   - Alternate pin functions reserved for hardware revisions

3.2.3 Communication Bandwidth Analysis

SPI Configuration: 10 Mbps is Optimal

Peak bandwidth calculation for RPi5 $\leftrightarrow$ RX72N communication:

- **Velocity Commands (100 Hz):**
  - Message size: 34 bytes (Nanopb encoded MotorVelocityCommand)
  - Bandwidth: 34 bytes $\times$ 100 Hz $\times$ 8 = 27.2 Kbps
- **Encoder Feedback (100 Hz):**
  - Message size: 18 bytes (4 encoders $\times$ 4 bytes + header)
  - Bandwidth: 18 bytes $\times$ 100 Hz $\times$ 8 = 14.4 Kbps
- **Telemetry (50 Hz):**
  - Message size: 64 bytes (battery, motor current, temperature, status)
  - Bandwidth: 64 bytes $\times$ 50 Hz $\times$ 8 = 25.6 Kbps
- **ARQ Protocol Overhead (estimated 40%):**
  - Headers: 8 bytes per frame (sequence, CRC-32, flags)
  - Retransmissions: Average 5% packet loss $\times$ 3 retry attempts
  - Total overhead: $(27.2 + 14.4 + 25.6) \times 1.4 = 94.1$ Kbps

Total Peak Bandwidth:

$$\text{Peak} = 27.2 + 14.4 + 25.6 + 94.1 = \mathbf{161.3\ Kbps}$$

SPI Utilization:

$$\frac{161.3\ \text{Kbps}}{10\ \text{Mbps}} = \mathbf{1.6\%\ utilization}$$

Margin Analysis:

- Available headroom: $10,000/161.3 = 62\times$ current requirements
- No need for Ethernet (would add PHY chip, 9+ pins, PCB complexity)
- No need for QSPI (RPi5 doesn't support quad-lane SPI natively)
- 10 Mbps provides excellent margin for future features (camera data, additional sensors)

3.3 PMOD Expansion Connectors

3.3.1 Overview

The STAR platform includes four standard 12-pin PMOD (Peripheral Module) connectors for maximum expandability. PMODs are standardized interfaces developed by Digilent for easy peripheral connectivity without requiring external bus capability (SRAM/Flash). All 32 unused GPIO pins are allocated to PMOD connectors.

PMOD Standard:

- 12-pin dual-row header (2x6, 0.1" pitch)
- Pins 5, 11: GND
- Pins 6, 12: VCC (3.3V, optionally 5V with level shifters)
- Pins 1-4, 7-10: Configurable signals (GPIO, SPI, I2C, UART, PWM, etc.)
- Compatible with 200+ commercial Digilent PMOD modules

3.3.2 PMOD JA - High-Speed SPI/Display Interface

Table 13: PMOD JA Pinout (High-Speed SPI)

Pin	Signal	RX72N	Function	Alternate Peripherals
1	CS0	P52	SPI Chip Select 0	RD#, SCI2 RX
2	COPI	P50	SPI Controller Out	WR#, SCI2 TX
3	CIPO	P51	SPI Controller In	WR1#/BC1#/WAIT#
4	SCK	P53	SPI Clock	BCLK (bus clock)
5	GND	-	Ground	-
6	VCC	-	3.3V Power	-
7	CS1	PB5	SPI Chip Select 1	A13, SCI9 SCK
8	DC	PB4	Data/Command	A12, Display control
9	RST	PB3	Reset	A11, SCI6 SCK
10	GPIO	PB2	Extra GPIO	A10
11	GND	-	Ground	-
12	VCC	-	3.3V Power	-

Compatible Modules:

- PMOD OLED (128x32 monochrome display)
- PMOD OLEDRGB (96x64 RGB display)
- PMOD SD (SD card reader)

- PMOD ALS (Ambient light sensor)
- PMOD ACL2 (3-axis accelerometer via SPI)

3.3.3 PMOD JB - I2C Sensor Hub

Table 14: PMOD JB Pinout (I2C Interface)

Pin	Signal	RX72N	Function	Notes
1	SCL	PC2	I2C Clock	RIIC2 capable, A18 bus conflict
2	SDA	PC3	I2C Data	RIIC2 capable, A19 bus conflict
3	INT1	PC4	Interrupt 1	IRQ-capable, A20/CS3#
4	INT2	PC5	Interrupt 2	IRQ-capable, A21/CS2#/WAIT#
5	GND	-	Ground	-
6	VCC	-	3.3V Power	-
7	GPIO0	PC6	Enable/Reset	A22/CS1#, D2 bus conflict
8	GPIO1	P55	General I/O	D0, WAIT#, EDREQ0
9	GPIO2	P54	General I/O	ALE, D1, EDACK0
10	GPIO3	PA2	General I/O	**Bus-free!** GTIOC1A alt
11	GND	-	Ground	-
12	VCC	-	3.3V Power	-

Compatible Modules:

- PMOD NAV (MPU-9250 9-DOF IMU)
- PMOD CMPS2 (HMC5883L 3-axis compass)
- PMOD TMP3 (TCN75A temperature sensor)
- PMOD AQS (iAQ-Core air quality sensor)
- PMOD HYGRO (HDC1080 humidity/temperature)
- Any I2C sensor breakout board

3.3.4 PMOD JC - UART/Wireless Interface

Table 15: PMOD JC Pinout (UART + Motor Driver SPI)

Pin	Signal	RX72N	Function	Alternate Peripherals
1	UART_TX	PD7	Transmit	D7, SCI12 TX, GLCDC
2	UART_RX	PD6	Receive	D6, SCI12 RX, GLCDC
3	RTS	PD5	Request to Send	D5, SCI12 flow control, QSPI
4	CTS	PD4	Clear to Send	D4, SCI12 flow control, QSPI
5	GND	-	Ground	-
6	VCC	-	3.3V/5V Power	Can be 5V with regulator
7	MOTOR_SCLK	PD3	Motor Driver SPI Clock	RSPCKC-A (repurposed from GPIO0)
8	MOTOR_CIPO	PD2	Motor Driver SPI In	MISOC-A (repurposed from GPIO1)
9	GPIO2	PB1	Extra GPIO	A9, LCD control
10	GPIO3	PB0	Extra GPIO	A8, LCD control
11	GND	-	Ground	-
12	VCC	-	3.3V/5V Power	-

Note: Pins 7 and 8 (PD3, PD2) have been repurposed for motor driver SPI configuration. PMOD JC still provides UART with flow control on pins 1-4 and two GPIO pins (9-10). For modules requiring GPIO0/GPIO1, use PMOD JA or JB instead.

Compatible Modules:

- PMOD GPS (u-blox NEO-6M/7M GPS)
- PMOD BT2 (Roving Networks RN-42 Bluetooth)
- PMOD ESP32 (WiFi + Bluetooth module)
- PMOD USBUART (USB to UART bridge)
- XBee modules (with adapter)
- LoRa modules (RFM95W with breakout)

3.3.5 PCB Layout Recommendations

Connector Placement:

- Place PMOD JA (SPI) and JB (I2C) on top edge of board for sensor access
- Place PMOD JC (UART) on bottom edge for wireless modules
- Maintain minimum 0.5" spacing between connectors for module clearance
- Orient all connectors with Pin 1 in same direction for consistency

Circuit Protection:

- **I2C Pull-ups (JB):** 2.2kΩ to 4.7kΩ resistors on SCL/SDA
- **Series Resistors:** 33Ω on all signal lines for ESD protection

- **Power Filtering:** 0.1μF ceramic + 10μF tantalum per connector
- **5V Option (JC):** TPS62133 buck converter or TLV62569 for 5V wireless modules
- **Overcurrent:** Resetable PTC fuses (500mA) on VCC lines

Trace Routing:

- **JA (High-Speed SPI):** 50Ω impedance-controlled traces, length match ±5mm
- **JB (I2C):** Keep traces <6 inches, avoid routing near PWM signals
- **JC (UART):** Cross TX/RX for DTE ↔ DCE direct connection

3.4 Motor Driver Dual-Interface Architecture

The DRV8243S motor drivers use two separate interfaces:

Real-Time Control (PWM):

- **Signals:** PH (Phase) and EN (Enable) on PE0-PE7
- **Purpose:** Motor speed/direction control at 20 kHz PWM frequency
- **Update rate:** 100 Hz velocity commands from RPi5

Configuration (SPI):

- **Peripheral:** RSPIC using PMOD JC GPIO pins (PD3, PD2, PD1, PA0, P17, P23, P32)
- **Purpose:** Configure current limits, fault thresholds, slew rates, dead-time insertion
- **Update rate:** Startup only, or when changing operating parameters
- **Speed:** 1 MHz SPI clock (configuration is not time-critical)
- **Signals:** SCLK (PD3), COPI (PD1), CIPO (PD2), nCS0 (P17), nCS1 (P23), nCS2 (P32), nCS3 (PA0)

Design Rationale:

- PWM provides low-latency real-time control for motor speed/direction
- SPI allows runtime configuration of driver protection features
- Separation of concerns: high-frequency control vs. low-frequency configuration
- Uses RSPIC to keep RSPIA available for RPi5 communication

3.5 RX72N Configuration Tables

Table 16: Emulator Configuration Jumper Settings

Jumper Position	**Configuration**
Shorted Pin 1-2 (normal Operation)	E1/E2 Lite normal debugging (use this mode for normal operation). Microcontroller single operation (without emulator)
Shorted Pin 2-3 (debug with E1/E2)	E1/E2 Lite debugging with Hot plug-in
All open	DO NOT SET

Table 17: Operation Configuration Mode DIP Switch Settings

Note: There are 2 USB-C ports. To flash we can use the SCI boot mode if we want to flash with the SCI USB-C. To flash we can use the USB0 boot mode if we want to flash with the USB0 USB-C.

Pin1	**Pin2**	**UPSEL_CFG**	**OP Mode**
OFF	X	X	Single Chip Mode
ON	OFF	X	SCI Boot Mode (SCI1)
ON	ON	1/2	USB (Bus Powered) (USB0)
ON	ON	2/3	USB (Self Powered) (USB0)

Table 18: UPSEL Power Configuration for USB Boot Mode

Jumper Position	**Power Configuration for USB Boot Mode**
Shorted Pin 1-2 (No battery pack)	Bus Powered
Shorted Pin 2-3 (Battery Pack)	Self Powered

Table 19: Required Pin States Upon Reset Release

To enter a mode capable of flashing firmware, the following pin configurations must be held when the **RES# pin** goes from Low to High:

Desired Flashing Mode	**MD Pin Level**	**UB Pin Level**	**Interface Used**
Boot Mode (SCI)	Low	Low	Serial (SCI1)
Boot Mode (USB)	Low	High	USB
Boot Mode (FINE)	Low → High*	Low	FINE Interface

*For FINE interface, the MD pin must be Low at reset release and then switched to High within 20 to 100 ms.

Table 20: SCI9 Firmware Configuration (MPC Settings)

Note: Pins are multiplexed. Firmware must explicitly configure the Multi-Function Pin Controller (MPC) as follows:

Step	Configuration
1. UNLOCK MPC	Unlock registers using the Write-Protect Register (PWPR)
2. PIN FUNCTION SELECT	**PIN 53 (PB7) → TXD9:** Set PB7PFS.PSEL[5:0] = 001010b (0x0A) **PIN 54 (PB6) → RXD9:** Set PB6PFS.PSEL[5:0] = 001010b (0x0A)
3. ENABLE PERIPHERAL	Set Port Mode Register (PMR) bit to '1' for both PB7 and PB6

USB-UART IC Configuration: Use Cypress configuration utility to enable BUSDETECT.

3.6 Raspberry Pi 5 Pinout

Status: Pin assignments are pending hardware verification and will be documented once validated.

3.7 Additional Peripheral Connections

Status: Peripheral connections are pending hardware verification and will be documented once validated.

4 Protocol Buffer Style Guide

This section defines naming standards and conventions for all Protocol Buffer definitions in the STAR project, following Boston Dynamics style guidelines.

4.1 Terminology Standard

The project uses inclusive terminology following OSHWA (Open Source Hardware Association) standards.

Table 21: Communication Bus Terminology

Use	Avoid	Context
Controller/Peripheral	Master/Slave	I2C, SPI, 1-Wire bus roles
COPI	MOSI	Controller Out, Peripheral In
CIPO	MISO	Controller In, Peripheral Out
Primary/Main	Master	Configuration structures

4.2 Protocol Buffer Naming Conventions (Boston Dynamics Style)

The STAR project follows Boston Dynamics' Protocol Buffer style guide for all .proto definitions.

Table 22: Protocol Buffer Naming Conventions

Element	Convention	Example
Package	Versioned namespace	`star.v1`
Messages	PascalCase	`VelocityCommand`, `BatteryState`
Fields	snake_case + unit suffix	`velocity_mps`, `temperature_celsius`
Enums	SCREAMING_SNAKE_CASE	`MOTOR_STATE_RUNNING`
Enum Zero Value	_UNKNOWN suffix	`STATUS_UNKNOWN = 0`
Enum Values	Type name prefix	`ROBOT_MODE_MANUAL`
RPC Pattern	{Verb}Request/Response	`SetVelocityRequest`

4.3 Unit Suffixes (MKS System)

All numeric fields use SI units with explicit suffixes. This eliminates unit ambiguity and enables automatic documentation generation.

Table 23: Standard Unit Suffixes

Suffix	Unit	Proto Example	C Example
`_m`	meters	`x_m`	`distance_m`
`_mps`	meters/second	`velocity_mps`	`speed_mps`
`_mps2`	m/s^2	`accel_x_mps2`	`acceleration_mps2`
`_rad`	radians	`angle_rad`	`heading_rad`
`_rad_per_s`	rad/second	`omega_rad_per_s`	`angular_vel_rad_per_s`
`_celsius`	degrees C	`temperature_celsius`	`motor_temp_celsius`
`_deci_celsius`	0.1°C	`temp_deci_celsius`	(BMS native format)
`_us`	microseconds	`timestamp_us`	`elapsed_us`
`_ms`	milliseconds	`timeout_ms`	`period_ms`
`_ma`	milliamps	`current_ma`	`motor_current_ma`
`_mv`	millivolts	`cell_mv`	`pack_voltage_mv`
`_mw`	milliwatts	`power_mw`	`consumption_mw`
`_percent`	0–100%	`soc_percent`	`duty_cycle_percent`

4.4 General Documentation Standards

- Use Doxygen-compatible comments for all public APIs
- Document the "why" not just the "what"
- Keep configuration centralized in header files
- Avoid magic numbers – use named constants with comments
- Version hardware and firmware together using git tags

4.5 NASA Power of 10: Rules for Safety-Critical Code

This section documents the NASA/JPL Power of 10 coding rules developed by Gerard J. Holzmann in 2006 for safety-critical embedded systems. These rules prioritize verification and predictability over coding flexibility.

STAR Compliance Philosophy: The STAR project follows the Power of 10 rules, with one **intentional architectural deviation** for Dependency Inversion Principle (DIP) — a conscious trade-off for testability and modularity.

4.5.1 Compliance Status Legend

Symbol	Meaning
✓ COMPLIANT	STAR follows this rule
△! INTENTIONAL DEVIATION	Architectural choice (DIP pattern)

4.5.2 Rule 1: Simplify Control Flow

Rule: Restrict all code to very simple control flow constructs — no `goto`, `setjmp`/`longjmp`, or recursion (direct or indirect).

Rationale: Creates predictable program structure for human understanding and static analysis. Recursion causes unbounded stack usage — dangerous in embedded systems.

STAR Compliance: ✓ COMPLIANT

```
/* COMPLIANT - No goto, no recursion */
esp_err_t star_motor_init(star_motor_handle_t* handle,
                          const star_motor_config_t* config)
{
  if (handle == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
  }

  /* All control flow uses if/while/for */
  esp_err_t ret = configure_timer(handle, config);
  if (ret != ESP_OK) {
    return ret;
  }

  return configure_gpio(handle, config);
}
```

4.5.3 Rule 2: Fixed Loop Upper-Bounds

Rule: All loops must have a fixed upper bound that static analysis tools can trivially prove.

Rationale: Prevents runaway code, guarantees termination, enables real-time deadline verification.

STAR Compliance: ✓ COMPLIANT

```
/* COMPLIANT - Fixed iteration count */
#define MAX_RETRIES (3)

for (uint8_t i = 0; i < MAX_RETRIES; i++) {
  if (sensor_read() == ESP_OK) {
    break;
  }
  vTaskDelay(pdMS_TO_TICKS(100));
}

/* COMPLIANT - Array with known size */
#define BQ7850_MAX_CELLS (16)

for (uint8_t cell = 0; cell < BQ7850_MAX_CELLS; cell++) {
  read_cell_voltage(cell, &voltages[cell]);
}
```

4.5.4 Rule 3: No Dynamic Memory After Initialization

Rule: Prohibit `malloc`/`free` after initialization phase.

Rationale: Dynamic allocation causes unpredictable performance, memory leaks, fragmentation, and corruption.

STAR Compliance: ✓ COMPLIANT

Best Practice: Use pre-allocated static pools instead of runtime memory requests.

```
/* COMPLIANT - Static allocation with fixed limits */
#define MAX_ITEMS (8)
```

```c
#define MAX_DESC_LEN (64)

typedef struct {
  char items[MAX_ITEMS][MAX_DESC_LEN];  /* Static 2D array */
  uint8_t item_count;
} static_pool_t;

/* Usage */
if (pool->item_count >= MAX_ITEMS) {
  return ESP_ERR_NO_MEM;
}
strncpy(pool->items[pool->item_count], desc, MAX_DESC_LEN - 1);
pool->items[pool->item_count][MAX_DESC_LEN - 1] = '\0';
pool->item_count++;
```

4.5.5 Rule 4: Keep Functions Short

Rule: Functions should not exceed ~60 lines (one printed page, one statement per line).

Rationale: Represents single verifiable logical units; improves comprehension, review, and testability.

STAR Compliance: ✓ MOSTLY COMPLIANT

```c
/* COMPLIANT - Single responsibility, short function */
esp_err_t star_drv8243_read_current(star_drv8243_handle_t* handle,
                                    float* current_ma)
{
  if (handle == NULL || current_ma == NULL || !handle->initialized) {
    return ESP_ERR_INVALID_ARG;
  }

  int raw_value = 0;
  esp_err_t ret = read_adc_channel(handle, &raw_value);
  if (ret != ESP_OK) {
    return ret;
  }

  *current_ma = (float)raw_value * handle->ki_propi / 1000.0f;
  return ESP_OK;
}
```

4.5.6 Rule 5: Use Assertions Generously

Rule: Minimum two assertions per function; assertions must be side-effect-free.

Rationale: Acts as executable documentation verifying pre-conditions, post-conditions, and invariants.

STAR Compliance: ✓ COMPLIANT

Note: STAR uses explicit parameter validation instead of assertions for production code (assertions disabled in release builds).

```c
/* COMPLIANT - Explicit validation (better than assert for production) */
esp_err_t star_pid_compute(star_pid_handle_t* handle, float setpoint,
                           float measurement, float dt, float* output)
```

```
{
  /* Pre-condition checks (effectively runtime assertions) */
  if (handle == NULL) {
    return ESP_ERR_INVALID_ARG;
  }
  if (!handle->initialized) {
    return ESP_ERR_INVALID_STATE;
  }
  if (output == NULL) {
    return ESP_ERR_INVALID_ARG;
  }
  if (dt <= 0.0f || dt > 1.0f) {  /* Sanity check time delta */
    return ESP_ERR_INVALID_ARG;
  }

  /* Function logic */
  float error = setpoint - measurement;
  handle->integral += error * dt;

  /* Post-condition: integral within bounds */
  if (handle->integral > handle->integral_max) {
    handle->integral = handle->integral_max;
  } else if (handle->integral < handle->integral_min) {
    handle->integral = handle->integral_min;
  }

  *output = handle->kp * error + handle->ki * handle->integral;
  return ESP_OK;
}
```

4.5.7 Rule 6: Declare Data at Smallest Scope

Rule: Minimize variable scope to reduce accidental corruption risks.

Rationale: Limits chances of accidental reference or corruption from other code.

STAR Compliance: ✓ **COMPLIANT**

```
/* COMPLIANT - Variables declared at minimal scope */
void process_sensors(void)
{
  /* Loop variable declared in for statement */
  for (uint8_t i = 0; i < SENSOR_COUNT; i++) {
    /* Temporary variable for single sensor */
    float temp_c = 0.0f;
    if (read_sensor(i, &temp_c) == ESP_OK) {
      /* Process immediately, temp_c out of scope after block */
      update_telemetry(i, temp_c);
    }
  }
}
```

4.5.8 Rule 7: Check All Return Values and Parameters

Rule: Validate all function returns and input parameters; treat warnings as errors.

Rationale: Forces explicit failure-condition handling rather than ignoring error codes.

STAR Compliance: ✓ COMPLIANT

```c
/* COMPLIANT - All returns checked, all parameters validated */
esp_err_t star_bus_manager_add_bus(star_bus_manager_t* manager,
                                   star_bus_config_t* config)
{
  /* Parameter validation */
  if (manager == NULL || config == NULL) {
    return ESP_ERR_INVALID_ARG;
  }

  /* Check return value and propagate errors */
  esp_err_t ret = star_bus_config_init(config);
  if (ret != ESP_OK) {
    ESP_LOGE(TAG, "Bus initialization failed: %s", esp_err_to_name(ret));
    return ret;
  }

  /* Mutex operation checked */
  if (xSemaphoreTake(manager->mutex, pdMS_TO_TICKS(1000)) != pdTRUE) {
    ESP_LOGE(TAG, "Failed to acquire mutex");
    return ESP_ERR_TIMEOUT;
  }

  /* Critical section */
  add_to_list(manager, config);
  xSemaphoreGive(manager->mutex);

  return ESP_OK;
}
```

4.5.9 Rule 8: Limit Preprocessor Use

Rule: Restrict to header inclusion and simple macros; forbid token pasting, recursive macros, and incomplete syntactic units.

Rationale: Preprocessor complexity obscures operations, fooling developers and static analysis tools.

STAR Compliance: ✓ COMPLIANT

```c
/* COMPLIANT - Simple constant macros */
#define MAX_RETRY_COUNT (3)
#define MOTOR_PWM_FREQ_HZ (20000)
#define TIMEOUT_MS (1000)

/* COMPLIANT - Simple inline functions (better than complex macros) */
static inline float voltage_mv_to_v(uint16_t mv)
{
  return (float)mv / 1000.0f;
}

/* AVOID - Complex token-pasting macros */
// #define CREATE_HANDLER(name) handler_##name##_init()  /* Too complex */
```

4.5.10 Rule 9: Restrict Pointer Use

Rule: Maximum one dereferencing level (`*ptr` OK, `**ptr` forbidden); no function pointers.

Rationale: Multiple indirection and function pointers make static data-flow analysis impossible.

STAR Compliance: △! INTENTIONAL DEVIATION

Intentional Architectural Deviation: STAR uses function pointers for **Dependency Inversion Principle (DIP)** — an industry-standard pattern for testability and modularity. This is a conscious trade-off for better architecture.

DIP Pattern (Intentional Function Pointer Use):

```
/* INTENTIONAL DEVIATION - DIP interface with function pointers */
typedef struct star_error_interface {
  esp_err_t (*record_error)(void* ctx, esp_err_t error, const char*
     message,
                            const char* file, int line, const char* func);
  bool (*can_retry)(void* ctx);
  esp_err_t (*reset_state)(void* ctx);
  void* ctx;  /* Implementation context */
} star_error_interface_t;

/* Why we keep this:
 * 1. Enables mock implementations for unit testing
 * 2. Allows swapping error handlers without recompilation
 * 3. Follows SOLID principles (Dependency Inversion)
 * 4. Industry-standard pattern (used in Linux kernel, RTOS, etc.)
 */
```

4.5.11 Rule 10: Compile with Maximum Warnings

Rule: Enable all compiler warnings at highest pedantry; achieve zero-warning builds with static analyzers.

Rationale: Modern compilers catch bugs before runtime. Zero warnings, no excuses.

STAR Compliance: ✓ COMPLIANT

```
# platformio.ini - Compiler flags
build_flags =
  -Wall              # Enable all warnings
  -Wextra            # Extra warnings
  -Werror            # Treat warnings as errors
  -Wno-unused-parameter  # Allow unused params in interface
     implementations
  -Wno-missing-field-initializers
```

CI/CD Enforcement: Build fails on any compiler warning.

4.5.12 Summary: STAR Compliance Matrix

Table 24: NASA Power of 10 Compliance Status

Rule	Description	STAR Status
1	No `goto`/recursion	✓ COMPLIANT
2	Fixed loop bounds	✓ COMPLIANT
3	No dynamic memory	✓ COMPLIANT
4	Short functions (<60 lines)	✓ COMPLIANT
5	Use assertions	✓ COMPLIANT
6	Minimal variable scope	✓ COMPLIANT
7	Check all returns	✓ COMPLIANT
8	Limit preprocessor	✓ COMPLIANT
9	Restrict pointers	△! INTENTIONAL (DIP)
10	Maximum warnings	✓ COMPLIANT

4.5.13 References

- Holzmann, Gerard J. *"The Power of 10: Rules for Developing Safety-Critical Code."* IEEE Computer, 39(6), pp. 95-97, June 2006.
- JPL Institutional Coding Standard for the C Programming Language (JPL DOCID D-60411)
- MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems

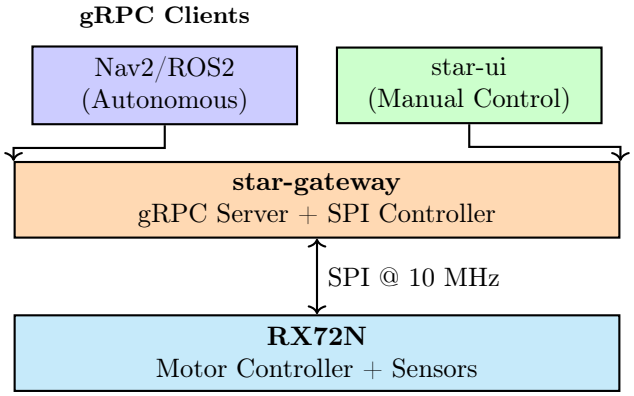
5 Gateway Architecture

This section documents the `star-gateway` Go service that bridges the Raspberry Pi 5 control stack with the RX72N motor controller.

5.1 Overview

The gateway service runs on the Raspberry Pi 5 and serves as the communication bridge between:

- **High-level side:** gRPC clients (Nav2/ROS2 for autonomous navigation, UI for manual control)
- **Low-level side:** RX72N motor controller via SPI at 10 MHz



5.2 Protocol Stack Implementation

The gateway implements Layers 1–4 of the communication protocol (see Section 1.2.1).

Table 25: Gateway Protocol Stack Implementation

Layer	Name	Go Package	Status
5	Application	`internal/service/`	Placeholder
4	Serialization	`star-proto/gen/go/`	Generated
3	ARQ	`internal/arq/`	Placeholder
2	Framing	`internal/frame/`	Implemented
1	Transport	`internal/transport/`	Placeholder

5.3 Directory Structure

```
star-gateway/
+-- cmd/
|   +-- star-gateway/
|       +-- main.go              # Entry point
+-- internal/
|   +-- transport/               # Layer 1: SPI transport
|   |   +-- spi.go               # Transport interface
|   |   +-- spi_test.go
|   +-- frame/                   # Layer 2: Frame protocol
|   |   +-- frame.go             # Constants and types
```

```
|   |   +-- encoder.go           # Frame encoder
|   |   +-- decoder.go           # Frame decoder
|   |   +-- frame_test.go
|   +-- arq/                     # Layer 3: ARQ protocol
|   |   +-- arq.go               # Stop-and-Wait ARQ
|   |   +-- arq_test.go
|   +-- service/                 # Layer 5: gRPC services
|       +-- motor_control.go
|       +-- telemetry.go
|       +-- battery.go
|       +-- configuration.go
|       +-- firmware.go
+-- go.mod
+-- CLAUDE.md
```

5.4 Frame Package

The `internal/frame` package defines the wire protocol constants and types.

5.4.1 Frame Constants

Table 26: Frame Protocol Constants

Constant	Value	Description
`SyncWord`	`0x55AA`	Frame sync marker (alternating bits: `[0x55, 0xAA]`)
`SyncSize`	2 bytes	Sync word size
`HeaderSize`	6 bytes	SEQ(2) + LEN(2) + TYPE(1) + FLAGS(1)
`CRCSize`	4 bytes	CRC-32 checksum
`MaxPayloadSize`	1024 bytes	Maximum protobuf payload
`MaxFrameSize`	1036 bytes	Total frame size
`MinFrameSize`	12 bytes	Empty payload frame

5.4.2 Frame Types

- `FrameTypeCommand` – Command from controller to peripheral
- `FrameTypeResponse` – Response from peripheral to controller
- `FrameTypeAck` – Acknowledgment of successful receipt
- `FrameTypeNack` – Negative acknowledgment (CRC error, etc.)

5.5 Transport Package

The `internal/transport` package provides the SPI transport layer.

5.5.1 SPI Configuration

Table 27: SPI Configuration Constants

Parameter	Value	Description
Device	`/dev/spidev0.0`	SPI device path
Speed	10 MHz	SPI clock frequency
Mode	0	CPOL=0, CPHA=0
Bits per word	8	Word size
Timeout	100 ms	Default operation timeout

5.5.2 Transport Interface

```
type Transport interface {
    Send(data []byte) (int, error)
    Receive(maxLen int) ([]byte, error)
    Transfer(txData []byte) ([]byte, error)
    Close() error
}
```

5.6 ARQ Package

The `internal/arq` package implements Stop-and-Wait ARQ for reliable delivery.

5.6.1 ARQ Parameters

Table 28: ARQ Protocol Parameters

Parameter	Value	Description
Max Retries	3	Maximum transmission attempts
Timeout	10 ms	ACK wait timeout
Sequence Max	0xFFFF	16-bit wraparound

5.6.2 ARQ State Machine

- **IDLE** – Ready to send or receive
- **WAITING_ACK** – Waiting for acknowledgment
- **RETRANSMITTING** – Retransmitting after timeout/NACK
- **ERROR** – Max retries exceeded

5.7 gRPC Services

The gateway exposes five gRPC services (see Section 2.2 for detailed specifications):

Table 29: Gateway gRPC Services

Service	Purpose
MotorControlService	Differential drive control, encoder streaming
TelemetryService	IMU, GPS, system status
BatteryManagementService	BQ7850 BMS monitoring
ConfigurationService	Runtime parameters, NVS persistence

5.8 Build and Test

```
# Build the gateway binary
cd star-gateway
go build ./cmd/star-gateway

# Run all tests
go test ./...

# Run with verbose output
go test -v ./...

# Format code
go fmt ./...

# Static analysis
go vet ./...
```

5.9 Dependencies

The gateway depends on:

- `github.com/Locked-Inc/star-proto/gen/go` – Generated protobuf and gRPC code
- `google.golang.org/grpc` – gRPC framework
- `google.golang.org/protobuf` – Protocol Buffers runtime

The `go.work` file at the repository root enables workspace mode for seamless cross-module development:

```
go 1.21

use (
    ./star-proto/gen/go
    ./star-proto/tests/go
    ./star-gateway
)
```